```
import pandas as pd
import re

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
```

```
train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")
sample_submission = pd.read_csv("sample_submission.csv")

print("Train shape:", train_df.shape)
print("Test shape:", test_df.shape)
print("Sample submission shape:", sample_submission.shape)
```

```
Train shape: (7613, 5)
Test shape: (3263, 4)
Sample submission shape: (3263, 2)
```

```
print("\nTrain columns:")
print(train_df.columns)

print("\nFirst 5 rows:")
print(train_df.head())
```

```

Train columns:
Index(['id', 'keyword', 'location', 'text', 'target'], dtype='object')

First 5 rows:
   id keyword location                                               text  \
0   1     NaN      NaN  Our Deeds are the Reason of this #earthquake M...
1   4     NaN      NaN             Forest fire near La Ronge Sask. Canada
2   5     NaN      NaN  All residents asked to 'shelter in place' are ...
3   6     NaN      NaN  13,000 people receive #wildfires evacuation or...
4   7     NaN      NaN  Just got sent this photo from Ruby #Alaska as ...

   target
0       1
1       1
2       1
3       1
4       1
```

```
print("\nMissing values in train:")
print(train_df.isnull().sum())

print("\nMissing values in test:")
print(test_df.isnull().sum())
```

```

Missing values in train:
id             0
keyword       61
location    2533
text           0
```

```
target         0
dtype: int64

Missing values in test:
id             0
keyword       26
location    1105
text           0
dtype: int64
```

```
print("\nClass balance:")
print(train_df["target"].value_counts())

print("\nClass balance percentage:")
print(train_df["target"].value_counts(normalize=True))
```

```
Class balance:
target
0    4342
1    3271
Name: count, dtype: int64

Class balance percentage:
target
0    0.57034
1    0.42966
Name: proportion, dtype: float64
```

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"@\w+", "", text)
    text = re.sub(r"#", "", text)
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

```
train_df["clean_text"] = train_df["text"].apply(clean_text)
test_df["clean_text"] = test_df["text"].apply(clean_text)

print("\nOriginal text example:")
print(train_df["text"].iloc[0])

print("\nCleaned text example:")
print(train_df["clean_text"].iloc[0])
```

```
Original text example:
Our Deeds are the Reason of this #earthquake May ALLAH Forgive us all

Cleaned text example:
our deeds are the reason of this earthquake may allah forgive us all
```

```
X = train_df["clean_text"]
y = train_df["target"]

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
```

```
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

print("\nSplit sizes:")
print("Train:", len(X_train))
print("Validation:", len(X_val))
print("Test:", len(X_test))
```

```
Split sizes:
Train: 5329
Validation: 1142
Test: 1142
```

```
vectorizer = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2)
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_val_tfidf = vectorizer.transform(X_val)
X_test_tfidf = vectorizer.transform(X_test)

print("\nTF-IDF shapes:")
print("Train:", X_train_tfidf.shape)
print("Validation:", X_val_tfidf.shape)
print("Test:", X_test_tfidf.shape)
```

```
TF-IDF shapes:
Train: (5329, 5000)
Validation: (1142, 5000)
Test: (1142, 5000)
```

```
baseline_model = LogisticRegression(max_iter=1000)

baseline_model.fit(X_train_tfidf, y_train)
```

```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

```
val_pred = baseline_model.predict(X_val_tfidf)

print("\nValidation Results:")
print("Accuracy:", round(accuracy_score(y_val, val_pred), 4))
print("Precision:", round(precision_score(y_val, val_pred), 4))
```

```
print("Recall:", round(recall_score(y_val, val_pred), 4))
print("F1-score:", round(f1_score(y_val, val_pred), 4))

print("\nValidation Classification Report:")
print(classification_report(y_val, val_pred))

print("\nValidation Confusion Matrix:")
print(confusion_matrix(y_val, val_pred))
```

```
Validation Results:
Accuracy: 0.7995
Precision: 0.8149
Recall: 0.6904
F1-score: 0.7475

Validation Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.88      0.83       651
           1       0.81      0.69      0.75       491

    accuracy                           0.80      1142
   macro avg       0.80      0.79      0.79      1142
weighted avg       0.80      0.80      0.80      1142


Validation Confusion Matrix:
[[574  77]
 [152 339]]
```

```
test_pred = baseline_model.predict(X_test_tfidf)

print("\nTest Results:")
print("Accuracy:", round(accuracy_score(y_test, test_pred), 4))
print("Precision:", round(precision_score(y_test, test_pred), 4))
print("Recall:", round(recall_score(y_test, test_pred), 4))
print("F1-score:", round(f1_score(y_test, test_pred), 4))

print("\nTest Classification Report:")
print(classification_report(y_test, test_pred))

print("\nTest Confusion Matrix:")
print(confusion_matrix(y_test, test_pred))
```

```
Test Results:
Accuracy: 0.817
Precision: 0.8386
Recall: 0.7102
F1-score: 0.7691

Test Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.90      0.85       652
           1       0.84      0.71      0.77       490

    accuracy                           0.82      1142
   macro avg       0.82      0.80      0.81      1142
weighted avg       0.82      0.82      0.81      1142
```

```
Test Confusion Matrix:
[[585  67]
 [142 348]]
```

```
test_tfidf = vectorizer.transform(test_df["clean_text"])

final_predictions = baseline_model.predict(test_tfidf)

submission = pd.DataFrame({
    "id": test_df["id"],
    "target": final_predictions
})

submission.to_csv("baseline_submission.csv", index=False)

print("\nSubmission saved as:baseline_submission.csv")

print(submission.head())
```

```
Submission saved as:baseline_submission.csv
   id  target
0   0       1
1   2       0
2   3       1
3   9       1
4  11       1
```

WEEK3

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from collections import Counter

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
Device: cpu
```

```
# 1. Tokenization

def tokenize(text):
    return str(text).split()

print("Example text:")
print(X_train.iloc[0])

print("\nTokens:")
print(tokenize(X_train.iloc[0]))
```

```
Example text:
las vegas in top cities for redlight running fatalities

Tokens:
['las', 'vegas', 'in', 'top', 'cities', 'for', 'redlight', 'running', 'fatalities']
```

```
# 2. Build vocabulary from training data

counter = Counter()

for text in X_train:
    counter.update(tokenize(text))

max_vocab_size = 10000

word2idx = {
    "<PAD>": 0,
    "<UNK>": 1
}

for word, count in counter.most_common(max_vocab_size - 2):
    word2idx[word] = len(word2idx)

print("Vocabulary size:", len(word2idx))
```

```
Vocabulary size: 10000
```

```
# 3. Encode text and add padding

max_len = 40

def encode_text(text):
    tokens = tokenize(text)
    ids = []

    for token in tokens:
        ids.append(word2idx.get(token, word2idx["<UNK>"]))

    if len(ids) < max_len:
        ids = ids + [word2idx["<PAD>"]] * (max_len - len(ids))
    else:
        ids = ids[:max_len]

    return ids

X_train_ids = [encode_text(text) for text in X_train]
X_val_ids = [encode_text(text) for text in X_val]
X_test_ids = [encode_text(text) for text in X_test]

print("Encoded example:")
print(X_train_ids[0])
print("Length:", len(X_train_ids[0]))
```

```
Encoded example:
[2595, 2099, 4, 181, 1784, 10, 2100, 767, 412, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
Length: 40
```

```
# 4. Create PyTorch Dataset

class TweetDataset(Dataset):
    def __init__(self, texts, labels):
        self.texts = torch.tensor(texts, dtype=torch.long)
```

```
        self.labels = torch.tensor(labels.values, dtype=torch.long)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.texts[idx], self.labels[idx]
```

```
# 5. Create DataLoaders

train_dataset = TweetDataset(X_train_ids, y_train)
val_dataset = TweetDataset(X_val_ids, y_val)
test_dataset = TweetDataset(X_test_ids, y_test)

batch_size = 32

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print("Train batches:", len(train_loader))
print("Validation batches:", len(val_loader))
print("Test batches:", len(test_loader))
```

```
Train batches: 167
Validation batches: 36
Test batches: 36
```

```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
        super().__init__()

        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim,
            padding_idx=0
        )

        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            batch_first=True,
            bidirectional=True
        )

        self.dropout = nn.Dropout(0.3)

        self.fc = nn.Linear(hidden_dim * 2, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)

        output, (hidden, cell) = self.lstm(embedded)

        last_hidden = torch.cat((hidden[-2], hidden[-1]), dim=1)

        last_hidden = self.dropout(last_hidden)
```

```
        logits = self.fc(last_hidden)

        return logits
```

```
# 7. GRU model

class GRUClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
        super().__init__()

        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim,
            padding_idx=0
        )

        self.gru = nn.GRU(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        output, hidden = self.gru(embedded)

        last_hidden = hidden[-1]
        logits = self.fc(last_hidden)

        return logits
```

```
# 8. Training function
# We use CrossEntropyLoss because the model predicts 2 classes: 0 and 1.

def train_model(model, train_loader, val_loader, epochs=5):
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

    for epoch in range(epochs):
        model.train()
        train_loss = 0

        for texts, labels in train_loader:
            texts = texts.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

            logits = model(texts)
            loss = criterion(logits, labels)

            loss.backward()
            optimizer.step()
```

```
            train_loss += loss.item()

        model.eval()
        val_loss = 0
        correct = 0
        total = 0

        with torch.no_grad():
            for texts, labels in val_loader:
                texts = texts.to(device)
                labels = labels.to(device)

                logits = model(texts)
                loss = criterion(logits, labels)

                val_loss += loss.item()

                preds = torch.argmax(logits, dim=1)

                correct += (preds == labels).sum().item()
                total += labels.size(0)

        avg_train_loss = train_loss / len(train_loader)
        avg_val_loss = val_loss / len(val_loader)
        val_accuracy = correct / total

        print(
            f"Epoch {epoch+1}/{epochs} | "
            f"Train Loss: {avg_train_loss:.4f} | "
            f"Val Loss: {avg_val_loss:.4f} | "
            f"Val Accuracy: {val_accuracy:.4f}"
        )

    return model
```

```
# 9. Evaluation function

def evaluate_model(model, data_loader):
    model.eval()

    all_preds = []
    all_labels = []

    with torch.no_grad():
        for texts, labels in data_loader:
            texts = texts.to(device)
            labels = labels.to(device)

            logits = model(texts)

            probs = torch.softmax(logits, dim=1)
            preds = torch.argmax(probs, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    acc = accuracy_score(all_labels, all_preds)
    prec = precision_score(all_labels, all_preds)
```

```
    rec = recall_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds)

    print("Accuracy:", round(acc, 4))
    print("Precision:", round(prec, 4))
    print("Recall:", round(rec, 4))
    print("F1-score:", round(f1, 4))

    print("\nClassification Report:")
    print(classification_report(all_labels, all_preds))

    print("\nConfusion Matrix:")
    print(confusion_matrix(all_labels, all_preds))

    return acc, prec, rec, f1, all_preds
```

```
lstm_model = LSTMClassifier(
    vocab_size=len(word2idx),
    embedding_dim=64,
    hidden_dim=64,
    output_dim=2
)

print(lstm_model)

lstm_model = train_model(
    lstm_model,
    train_loader,
    val_loader,
    epochs=5
)
```

```
LSTMClassifier(
  (embedding): Embedding(10000, 64, padding_idx=0)
  (lstm): LSTM(64, 64, batch_first=True, bidirectional=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (fc): Linear(in_features=128, out_features=2, bias=True)
)
Epoch 1/5 | Train Loss: 0.6521 | Val Loss: 0.5975 | Val Accuracy: 0.6970
Epoch 2/5 | Train Loss: 0.5297 | Val Loss: 0.5234 | Val Accuracy: 0.7443
Epoch 3/5 | Train Loss: 0.4340 | Val Loss: 0.5165 | Val Accuracy: 0.7539
Epoch 4/5 | Train Loss: 0.3541 | Val Loss: 0.5496 | Val Accuracy: 0.7443
Epoch 5/5 | Train Loss: 0.2846 | Val Loss: 0.5611 | Val Accuracy: 0.7539
```

```
print("LSTM Test Results:")

lstm_accuracy, lstm_precision, lstm_recall, lstm_f1, lstm_preds = evaluate_model(
    lstm_model,
    test_loader
)
```

```
LSTM Test Results:
Accuracy: 0.7758
Precision: 0.7566
Recall: 0.7041
F1-score: 0.7294

Classification Report:
              precision    recall  f1-score   support
```

```
           0       0.79      0.83      0.81       652
           1       0.76      0.70      0.73       490

    accuracy                           0.78      1142
   macro avg       0.77      0.77      0.77      1142
weighted avg       0.77      0.78      0.77      1142


Confusion Matrix:
[[541 111]
 [145 345]]
```

```
# 12. Train GRU

gru_model = GRUClassifier(
    vocab_size=len(word2idx),
    embedding_dim=embedding_dim,
    hidden_dim=hidden_dim,
    output_dim=output_dim
)

print(gru_model)

gru_model = train_model(
    gru_model,
    train_loader,
    val_loader,
    epochs=5
)
```

```
GRUClassifier(
  (embedding): Embedding(10000, 64, padding_idx=0)
  (gru): GRU(64, 64, batch_first=True)
  (fc): Linear(in_features=64, out_features=2, bias=True)
)
Epoch 1/5 | Train Loss: 0.6838 | Val Loss: 0.6837 | Val Accuracy: 0.5701
Epoch 2/5 | Train Loss: 0.6835 | Val Loss: 0.6833 | Val Accuracy: 0.5701
Epoch 3/5 | Train Loss: 0.6506 | Val Loss: 0.6096 | Val Accuracy: 0.6830
Epoch 4/5 | Train Loss: 0.5486 | Val Loss: 0.5719 | Val Accuracy: 0.7250
Epoch 5/5 | Train Loss: 0.4349 | Val Loss: 0.5476 | Val Accuracy: 0.7539
```

```
# 13. Evaluate GRU

print("GRU Test Results:")

gru_accuracy, gru_precision, gru_recall, gru_f1, gru_preds = evaluate_model(
    gru_model,
    test_loader
)
```

```
GRU Test Results:
Accuracy: 0.7653
Precision: 0.7817
Recall: 0.6286
F1-score: 0.6968

Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.87      0.81       652
           1       0.78      0.63      0.70       490
```

```
    accuracy                           0.77      1142
   macro avg       0.77      0.75      0.75      1142
weighted avg       0.77      0.77      0.76      1142


Confusion Matrix:
[[566  86]
 [182 308]]
```

```
# 14. Compare baseline, LSTM, and GRU

comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression Baseline",
        "LSTM",
        "GRU"
    ],
    "Accuracy": [
        0.8170,
        lstm_accuracy,
        gru_accuracy
    ],
    "Precision": [
        0.8386,
        lstm_precision,
        gru_precision
    ],
    "Recall": [
        0.7102,
        lstm_recall,
        gru_recall
    ],
    "F1-score": [
        0.7691,
        lstm_f1,
        gru_f1
    ]
})

comparison
```

	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Regression Baseline	0.817000	0.838600	0.710200	0.769100
1	LSTM	0.775832	0.756579	0.704082	0.729387
2	GRU	0.765324	0.781726	0.628571	0.696833

Далее: Создать код с переменной `comparison` New interactive sheet

```
# 15. Error analysis

error_analysis = pd.DataFrame({
    "text": X_test.values,
    "true_label": y_test.values,
    "lstm_prediction": lstm_preds,
```

```
    "gru_prediction": gru_preds
})

lstm_errors = error_analysis[
    error_analysis["true_label"] != error_analysis["lstm_prediction"]
]

gru_errors = error_analysis[
    error_analysis["true_label"] != error_analysis["gru_prediction"]
]

print("Number of LSTM errors:", len(lstm_errors))
print("Number of GRU errors:", len(gru_errors))
```

```
Number of LSTM errors: 256
Number of GRU errors: 268
```

```
print("LSTM error examples:")
display(lstm_errors.head(10))
```

```
LSTM error examples:
```

	text	true_label	lstm_prediction	gru_prediction
1	on the flip side im at walmart and there is a ...	1	0	0
13	spot flood combo inch w curved cree led work l...	1	0	0
19	egyptian militants tied to isis threaten to ki...	1	0	0
21	unpredictable disconnected and social casualty...	1	0	0
23	auction shoes retro fire red	0	1	1
29	everyones wonder who will win and im over here...	0	1	0
30	turkish couple decided to feed syrian refugees...	0	1	0
32	looks like it may have been microsofts anti vi...	0	1	0
34	hollywood movie about trapped miners released ...	1	0	0
36	excessive engine failure rate significant main...	0	1	1

```
print("GRU error examples:")
display(gru_errors.head(10))
```

GRU error examples:

	text	true_label	lstm_prediction	gru_prediction
1	on the flip side im at walmart and there is a ...	1	0	0
6	i learned more about economics from one south ...	1	1	0
10	hwrf absolutely lashes taipei with hurricane f...	1	1	0
12	tube strike live latest travel updates as lond...	1	1	0
13	spot flood combo inch w curved cree led work l...	1	0	0
19	egyptian militants tied to isis threaten to ki...	1	0	0
21	unpredictable disconnected and social casualty...	1	0	0
23	auction shoes retro fire red	0	1	1
24	zayn malik amp perrie edwards end engagement s...	1	1	0
25	the xrays in mkx be looking like fatalities	0	0	1